



```
$ qa_cheatsheet --  
start
```

Cypress & Artillery

```
you@qa-engineer:~/automation$ cat README.md
```

Повна шпаргалка по автоматизованому тестуванню
на Cypress (E2E + API) і Artillery (Performance).
Простими словами, з прикладами і лайфхаками.

⚡ Cypress



Artillery



API Testing



Page Object



Performance

Made with ❤️ for QA Automation Engineers



Зміст

1. Що таке Cypress
2. Налаштування оточення
3. Установка та перший запуск
4. Структура проєкту
5. Структура тесту + хуки
6. Базові команди Cypress
7. Запуск у різних браузерах
8. Селектори і вебелементи
9. Assertions (твердження)
10. Події мишки
11. Власні команди
12. Page Object Pattern
13. API тестування — переваги
14. Перший API тест
15. Формування HTTP-запитів
16. Перевірка HTTP-відповіді
17. Debugging тестів
18. Просунуті техніки API
19. Artillery — вступ
20. Встановлення Artillery
21. Перший Artillery тест
22. POST та capture
23. Плагін ensure
24. Типи навантажувальних тестів
25. Додаткові можливості Artillery
26. 🎯 Міні-шпаргалка (Cypress + Artillery)
27. ✅ Чекліст QA (Cypress + Artillery)
28. Що таке CI/CD
29. CI/CD для AQA
30. Continuous Integration — етапи
31. Continuous Delivery
32. Інструменти CI/CD
33. Що таке BDD
34. Cucumber.io та Step Definitions
35. Gherkin — синтаксис
36. Встановлення Cucumber
37. Перший Cucumber тест
38. Структурні підходи (Outline, Tables)
39. 🎯 Міні-шпаргалка (CI/CD + BDD)
40. ✅ Чекліст (CI/CD + BDD)

Частина 1. Cypress – знайомство

1. Що таке Cypress

Cypress — це сучасний фреймворк для автоматизованого тестування вебдодатків. Простими словами — це інструмент, який «клікає за тебе» по сайту і перевіряє, що все працює.

→ Що під капотом

- ▶ Побудований на `Node.js`
- ▶ Усередині — фреймворки **Mocha**, **Chai**, **Jest**
- ▶ Тести пишуться на **JavaScript** або **TypeScript**
- ▶ Працює прямо в браузері (без Selenium / WebDriver)

→ Які тести можна писати

Тип	Що перевіряє
<code>End-to-end</code>	Весь шлях користувача від А до Я
<code>Component</code>	Окремі UI-компоненти
<code>Integration</code>	Взаємодію модулів між собою
<code>Unit</code>	Найменші частини коду (функції)

→ Топ фічі Cypress

- ▶ **Time Travel** — снапшоти на кожному кроці. Можна навести на крок і побачити, як виглядав інтерфейс
- ▶ **Auto Waiting** — Cypress сам чекає елементи. Жодних `sleep()`
- ▶ **Debuggability** — можна дебажити прямо в DevTools браузера
- ▶ **Screenshots & Video** — автоматично записує відео і скрін при падінні
- ▶ **Стабільність** — без Selenium, тому швидко і надійно

💡 LIFEHACK

Cypress — це «два-в-одному»: і фреймворк, і test runner. Не треба нічого додатково встановлювати, щоб запускати тести.

2. Налаштування робочого оточення

Перед стартом треба підготувати комп'ютер. Йдемо по кроках.

→ Крок 1. Браузери

Cypress працює з: **Chrome**, **Firefox**, **Edge**. Постав останні версії.

→ Крок 2. Node.js

Node.js — це середовище, яке вміє виконувати JavaScript на твоєму комп'ютері (а не тільки в браузері). Без нього Cypress не запрацює.

- ▶ Йди на `nodejs.org`
- ▶ Качай **LTS-версію** (стабільна)
- ▶ Постав через інсталятор

Перевірка встановлення

```
$ node -v
```

Якщо побачив щось типу `v20.11.0` — все ок 

→ Крок 3. Редактор коду + термінал

- ▶ **VSCode** — найпопулярніший редактор для JS/TS
- ▶ **Git Bash** (опційно) — зручний термінал для Windows

⚠ ТИПОВА ПОМИЛКА

Не використовуй Windows CMD — там бувають проблеми з кодуванням і командами. Бери **Git Bash** або вбудований термінал VSCode.

3. Установка та перший запуск Cypress

→ Крок 1. Ініціалізація проекту

```
$ npm init -y
```

Створює файл `package.json` з дефолтними налаштуваннями. Це паспорт твого проекту.

→ Крок 2. Встановлення Cypress

```
$ npm install cypress --save-dev
```

Прапорець `--save-dev` означає «це інструмент для розробки, а не для продакшену».

→ Крок 3. Запуск Cypress

```
$ npx cypress open
```

→ Що далі

1. Обери **E2E Testing**
2. Натисни **Continue** (Cypress сам створить конфігурацію)
3. Обери браузер (зазвичай Chrome)
4. Натисни **create new empty spec** → перший тест готовий

💡 LIFEHACK

Якщо `npx cypress open` не працює — закрий VSCode і відкрий його знову. VSCode інколи не «бачить» свіжовстановлені пакети.

4. Структура проєкту

Після першого запуску в тебе з'являться файли і папки. Розбираємось, хто за що відповідає.

Папка / Файл	Що це
<code>cypress/e2e</code>	Тут лежать твої тести (файли <code>*.cy.js</code>)
<code>cypress/fixtures</code>	Тестові дані (JSON-и з юзерами, продуктами тощо)
<code>cypress/support/commands.js</code>	Власні команди (custom commands)
<code>cypress/support/e2e.js</code>	Допоміжні налаштування для E2E
<code>node_modules</code>	Усі залежності проєкту (НЕ чіпати руками)
<code>cypress.config.js</code>	Конфіг Cypress (baseUrl, timeouts тощо)
<code>package.json</code>	Загальні налаштування проєкту
<code>package-lock.json</code>	Точні версії пакетів (для команди)

📌 НА ЗАМІТКУ

Папку `node_modules` ніколи не комітять у Git. Додай її в `.gitignore`.

Частина 2. Пишемо тести на Cypress

5. Структура тесту

Кожен тест на Cypress будується за однаковим шаблоном:

```
describe("Тестуємо шапку сайту", () => {  
  it("Перевіряємо логотип", () => {  
    // тут код тесту  
  });  
  it("Перевіряємо меню", () => {  
    // тут код тесту  
  });  
});
```

→ describe – група тестів

Об'єднує тести однієї теми. Приймає два аргументи:

- ▶ **Назва групи** — що тестуємо
- ▶ **Функція** — всередині будуть тести

→ it – окремий тест

- ▶ **Назва тесту** — що саме перевіряємо
- ▶ **Функція** — сам код перевірок

→ Хуки (Hooks) – підготовка і прибирання

Хук	Коли виконується	Для чого
<code>before()</code>	Один раз перед усіма тестами	Початкові налаштування
<code>beforeEach()</code>	Перед кожним тестом	Скинути стан, залогінитись
<code>afterEach()</code>	Після кожного тесту	Прибирання за тестом
<code>after()</code>	Один раз після всіх	Фінальне очищення
<code>.only()</code>	Тільки цей тест	Запустити один тест ізольовано
<code>.skip()</code>	Пропустити тест	Тимчасово вимкнути

🌟 **ЖИВИЙ ПРИКЛАД**

```
describe("Login tests", () => {
  beforeEach(() => {
    cy.visit("/login"); // перед кожним тестом
  });

  it.only("тільки цей тест запуститься", () => { });
  it.skip("цей пропуститься", () => { });
});
```

💡 LIFEHACK

Cypress сам ізолює тести: чистить cookies, localStorage, sessionStorage перед кожним `it()`. Тому тести не залежать один від одного.

⚠️ ТИПОВА ПОМИЛКА

Не залишай `.only()` у комітах! Інакше CI прожене лише один тест, а ти думатимеш, що пройшли всі. Class бойова помилка.

6. Базові команди Cypress

Тести імітують дії юзера. Для цього є команди. Вони діляться на два типи: **батьківські** і **команди-нащадки**.

→ Батьківські команди (починають ланцюжок)

Команда	Що робить	Приклад
<code>cy.visit()</code>	Відкриває сайт	<code>cy.visit("/login")</code>
<code>cy.get()</code>	Шукає елемент за селектором	<code>cy.get("#email")</code>
<code>cy.contains()</code>	Шукає елемент за текстом	<code>cy.contains("Submit")</code>

→ Команди-нащадки (йдуть після батьківських)

Команда	Що робить
<code>.click()</code>	Клік мишкою
<code>.type()</code>	Ввести текст
<code>.submit()</code>	Відправити форму
<code>.hover()</code>	Навести курсор

`.clear()`

Очистити поле

🌟 ЖИВИЙ ПРИКЛАД

```
// Логін за 4 рядки
cy.visit("/login");
cy.get("#email").type("user@test.com");
cy.get("#password").type("123456");
cy.get("[type='submit']").click();
```

7. Запуск тестів у різних браузерах

→ Запуск через GUI

```
$ npx cypress open
```

Відкриває візуальний інтерфейс. Зручно для розробки і дебагу.

→ Запуск через термінал (headless / headed)

```
$ npx cypress run -b chrome --headed
```

→ Розбір команди

- ▶ `npx cypress run` — запустити всі тести з папки `e2e`
- ▶ `-b chrome` — у якому браузері (chrome / firefox / edge)
- ▶ `--headed` — показати вікно браузера. Без нього — буде «фоновий» режим

💡 LIFEHACK

Після `cypress run` у папці `cypress/videos` з'являться відео тестів. Зручно для звітів і коли треба переглянути падіння.

→ Корисні прапорці

Прапорець	Що робить
<code>--spec "cypress/e2e/login.cy.js"</code>	Запустити лише один файл
<code>--browser firefox</code>	Те саме, що <code>-b firefox</code>
<code>--headless</code>	Без вікна (для CI)

8. Селектори і вебелементи

Щоб взаємодіяти з елементом, спочатку треба його **знайти**. Для цього використовують селектори.

→ Як знайти елемент на сторінці

1. Натисни `F12` — відкриються DevTools
2. Вкладка **Elements**
3. Натисни на іконку курсора (Inspect)
4. Клікни на елемент на сторінці

→ Типи селекторів

Селектор	Що означає	Приклад
<code>.class</code>	За класом	<code>cy.get(".btn-primary")</code>
<code>#id</code>	За унікальним ID	<code>cy.get("#user_email")</code>
<code>tag</code>	За тегом	<code>cy.get("input")</code>
<code>[attr=value]</code>	За атрибутом	<code>cy.get("[type=email]")</code>
<code>[data-cy=...]</code>	Кастомний атрибут (BEST!)	<code>cy.get("[data-cy=submit]")</code>

→ Чому варто використовувати `data-cy`

- ▶ Класи і теги змінюються — тести ламаються
- ▶ ID можуть генеруватися автоматично
- ▶ `data-cy` — стабільний атрибут **тільки для тестів**

🌟 ЖИВИЙ ПРИКЛАД

```
// HTML
<button data-cy="submit-btn">Submit</button>

// Cypress
cy.get('[data-cy="submit-btn"]').click();
```

⚠ ТИПОВА ПОМИЛКА

Не використовуй селектори типу `.css-1f1fv1i` або `div > div > span`. Це автогенеровані класи — вони зміняться завтра і тест впаде.

💡 LIFEHACK

Запусти Cypress у режимі `open` — там є **Selector Playground**. Клацни іконку «прицілу» і вибери елемент → Cypress сам згенерує селектор.

9. Assertions (твердження)

Assertion — це перевірка «чи відповідає елемент очікуванню». Простіше: «я ОЧІКУЮ, що тут буде ось так».

→ Основні методи

Метод	Призначення
<code>.should()</code>	Одна перевірка
<code>.and()</code>	Додати ще одну перевірку до тієї самої команди

→ Список найчастіших тверджень

Твердження	Що перевіряє
<code>be.visible</code>	Елемент видимий
<code>exist</code> / <code>not.exist</code>	Існує / не існує в DOM
<code>have.text</code>	Має такий текст
<code>have.value</code>	Інпут має таке значення
<code>have.class</code>	Має такий клас
<code>have.css</code>	Має таку CSS-властивість
<code>be.checked</code>	Чекбокс відмічений
<code>be.disabled</code>	Кнопка/інпут заблокований
<code>contain</code>	Містить підрядок

🌟 ЖИВИЙ ПРИКЛАД

```
// окремі should
cy.get('[type="submit"]').should("have.text", "Log in");
cy.get('[type="submit"]').should("have.css", "background-color", "rgb(255, 107, 10)");

// Або об'єднано через and (краще)
cy.get('[type="submit"]')
  .should("be.visible")
  .and("have.text", "Log in")
  .and("have.css", "background-color", "rgb(255, 107, 10)");
```

💡 LIFEHACK

Якщо потрібно зробити кілька перевірок над одним елементом — використовуй `.and()`. Так читабельніше і не дублюється `cy.get()`.

10. Події мишки

Користувач не лише клікає — він скролить, наводить, робить подвійний клік. Cypress це теж вміє.

Метод	Що імітує
<code>.click()</code>	Простий клік
<code>.dblclick()</code>	Подвійний клік
<code>.rightclick()</code>	Клік правою кнопкою
<code>.scrollIntoView()</code>	Проскролити до елемента
<code>.trigger("mouseover")</code>	Навести курсор (hover)

🌟 ЖИВИЙ ПРИКЛАД

```
it("scroll до футера", () => {
  cy.visit("/");
  cy.get('[id="footer"]')
    .scrollIntoView()
    .should("be.visible");
});
```

11. Власні команди (Custom Commands)

Якщо ти бачиш, що один і той самий код повторюється в багатьох тестах — винеси його в команду. Це принцип **DRY (Don't Repeat Yourself)**.

→ Де створювати

Файл `cypress/support/commands.js`

Syntax → `Cypress.Commands.add(name, callbackFn)`

🌟 ЖИВИЙ ПРИКЛАД

Замість того, щоб у кожному тесті писати 4 рядки логіну:

```
// cypress/support/commands.js
Cypress.Commands.add('signIn', (email, password) => {
  cy.get('#user_email').type(email);
  cy.get('#user_password').type(password);
  cy.get("[type='submit']").click();
});
```

І тепер у тестах просто:

```
it("admin login", () => {
  cy.visit("/login");
  cy.signIn("admin@test.com", "123456");
});

it("user login", () => {
  cy.visit("/login");
  cy.signIn("user@test.com", "qwerty");
});
```

💡 LIFEHACK

Замість 12 рядків коду — 6. А якщо щось зміниться в логіні — правиш в одному місці.

Частина 3. Page Object Pattern

12. Page Object Pattern (POP)

Page Object Pattern — це підхід, де код для роботи з кожною сторінкою виноситься в окремий клас. Замість «купи команд у тесті» — у тебе є чистий клас з методами.

→ Навіщо це потрібно

- ▶ Код стає **читабельний**: `LoginPage.signIn()` зрозуміліше за 5 рядків `cy.get + cy.type`
- ▶ Якщо змінилась верстка — правиш в **одному місці**, а не в 50 тестах
- ▶ Селектори і логіка не дублюються
- ▶ Тести стають коротшими

→ Як організувати папки

```
cyress/  
├─ e2e/  
│   └─ login.cy.js  
├─ pages/          ← створюємо нову папку  
│   ├── LoginPage.js  
│   ├── HomePage.js  
│   └─ CartPage.js  
└─ support/
```

→ Створюємо клас сторінки

★ ЖИВИЙ ПРИКЛАД

Файл `cyress/pages/LoginPage.js` :

```
export class Login {  
  navigate() {  
    cy.visit("/login");  
  }  
  
  validateLoginTitle() {  
    cy.get(".login-title")  
      .should("be.visible")  
      .and("have.text", "Login");  
  }  
  
  validateInputs() {  
    cy.get("#user_email").should("be.visible");  
    cy.get("#user_password").should("be.visible");  
  }  
  
  signIn(email, password) {  
    cy.get("#user_email").type(email);  
    cy.get("#user_password").type(password);  
    cy.get("[type='submit']").click();  
  }  
}
```

→ Використовуємо у тесті

🌟 ЖИВИЙ ПРИКЛАД

Файл `cypress/e2e/login.cy.js`:

```
import { Login } from "../pages/LoginPage";

const LoginPage = new Login();

describe("Login page tests", () => {
  it("відкриваємо сторінку і перевіряємо UI", () => {
    LoginPage.navigate();
    LoginPage.validateLoginTitle();
    LoginPage.validateInputs();
  });

  it("успішний логін", () => {
    LoginPage.navigate();
    LoginPage.signIn("user@test.com", "123456");
  });
});
```

→ Принципи ООП у ROR

- ▶ **Наслідування** — якщо хедер однаковий на 10 сторінках, створи `HeaderPage` і успадковуй від нього
- ▶ **Абстракція** — у тесті ти просто кажеш `LoginPage.signIn()`, не думаючи про деталі
- ▶ **Інкапсуляція** — селектори ховаються всередині класу

💡 LIFEHACK

Назви методів описують **дію**: `signIn()`, `fillForm()`, `clickSubmit()` — а не `method1()` чи `doStuff()`.

⚠ ТИПОВА ПОМИЛКА

Не клади асерти всередину «дієвих» методів типу `signIn()`. Розділяй: окремі методи для дій, окремі — для перевірок (`validateXxx()`).

Частина 4. API тестування на Cypress

13. Чому варто тестувати API

API-тести перевіряють «начинку» сайту — те, що ховається за UI. Це окремий шар тестування, який дає купу плюсів.

→ Переваги API-тестів

- ▶ Можна тестувати без UI — поки фронт ще пишеться
- ▶ Раннє виявлення багів — а отже, дешевше виправляти
- ▶ Стабільність — API змінюється рідше за UI
- ▶ Швидкість — у десятки разів швидше за E2E
- ▶ Покриття — можна послати **невалідні** дані, які UI не дасть надіслати
- ▶ Прекондиції для E2E — створив юзера/продукт через API за 1 секунду замість 30 кліків

💡 LIFEHACK

Гарний QA пише **піраміду тестів**: багато API-тестів унизу, трохи E2E зверху. Не навпаки.

14. Перший API-тест на Cypress

Для прикладів використаємо `httpbin.org` — спеціальний сайт для тренування API-тестів.

→ Найпростіший тест

🌟 ЖИВИЙ ПРИКЛАД

```
describe('httpbin tests', () => {
  it('response code should be 200', () => {
    cy.request('https://httpbin.org').then(response => {
      const status = response.status;
      assert.equal(200, status);
    });
  });
});
```

→ Розбираємо по рядках

- ▶ `cy.request(url)` — відправляє HTTP-запит

- ▶ `.then(response => {...})` — отримуємо відповідь
- ▶ `response.status` — код відповіді (200 = OK)
- ▶ `assert.equal(expected, actual)` — перевірка

→ Якщо очікуємо НЕ 200

За замовчуванням Cypress «падає» на будь-якому коді $\neq$ 200. Щоб тестувати помилки — додай

`failOnStatusCode: false`:

🌟 ЖИВИЙ ПРИКЛАД

```
const request = {
  url: 'https://httpbin.org/non-existing',
  failOnStatusCode: false
};

cy.request(request).then(response => {
  assert.equal(404, response.status);
});
```

15. Формуємо HTTP-запит

Будь-який запит = метод + URL + заголовки + тіло. Розбираємо все, що можна додати в об'єкт `request`.

→ HTTP-методи (CRUD)

Метод	Дія	Розшифровка
<code>GET</code>	Отримати дані	Read
<code>POST</code>	Створити	Create
<code>PUT</code> / <code>PATCH</code>	Оновити	Update
<code>DELETE</code>	Видалити	Delete

→ Об'єкт запиту — повна структура

🌟 ЖИВИЙ ПРИКЛАД

```
const request = {
  method: 'POST',
  url: 'https://httpbin.org/post',
  qs: { key: "value" },           // query-параметри
  headers: {
    "Authorization": "Bearer xxx",
    "Content-Type": "application/json"
  },
  body: {                         // тіло у JSON
    username: "test",
    password: "123"
  },
  failOnStatusCode: false
};

cy.request(request).then(response => {
  assert.equal(200, response.status);
});
```

→ Що таке JSON (швидко)

JSON — це формат обміну даними. Виглядає як JS-об'єкт: пари **ключ : значення**, у фігурних дужках.

```
{
  "name": "John",
  "age": 30,
  "isAdmin": true
}
```

→ Що зазвичай тестують в API

- ▶ **Нарру path** — все валідне, очікуємо 200 і коректну відповідь
- ▶ **Невалідні параметри** — порожні, неправильного типу, відсутні
- ▶ **Невалідні заголовки** — без токена, з простроченим токеном
- ▶ **Неправильний метод** — POST там, де чекає GET → 405

💡 LIFEHACK

Завжди починай з Нарру Path. Якщо він не працює — нема сенсу тестувати негативні кейси.

16. Перевірка HTTP-відповіді

Об'єкт `response` містить усе, що повернув сервер. Перевіряти можна різні частини.

→ Що є в `response`

Поле	Що містить
<code>response.status</code>	Код відповіді (200, 404, 500...)
<code>response.body</code>	Тіло відповіді (JS-об'єкт)
<code>response.headers</code>	Заголовки відповіді
<code>response.duration</code>	Час виконання (мс)
<code>response.requestHeaders</code>	Заголовки, які пішли в запиті

→ Перевірка коду

🌟 ЖИВИЙ ПРИКЛАД

```
cy.request(request).then(response => {  
  assert.equal(200, response.status);  
});
```

→ Перевірка конкретного поля

🌟 ЖИВИЙ ПРИКЛАД

```
cy.request('/api/users/2').then(resp => {  
  assert.equal("janet@reqres.in", resp.body.data.email);  
});
```

→ Перевірка через квадратні дужки

Якщо ключ містить дефіс, пробіл або починається з великої — без дужок не дістати:

🌟 ЖИВИЙ ПРИКЛАД

```
assert.equal("value", response.requestHeaders['Cookie']);
assert.equal("app/json", response.headers['content-type']);
```

17. Debugging API-тестів

Коли тест падає, треба зрозуміти що саме повернув сервер. Є три способи.

→ Спосіб 1. `console.log()`

🌟 ЖИВИЙ ПРИКЛАД

```
cy.request(request).then(resp => {
  console.log(resp);
  assert.equal(200, resp.status);
});
```

Дивитись у DevTools → Console

→ Спосіб 2. `cy.log()`

🌟 ЖИВИЙ ПРИКЛАД

```
cy.request(request).then(resp => {
  cy.log(resp.body.data.email);
});
```

Виводиться прямо в Test Runner Cypress

→ Спосіб 3. `debugger`

🌟 ЖИВИЙ ПРИКЛАД

```
cy.request(request).then(resp => {
  debugger;           // тут зупиниться
  const email = resp.body.data.email;
  assert.equal("test@x.com", email);
});
```

DevTools мають бути відкриті, інакше не зупиниться

💡 LIFEHACK

Для швидкого «що там у відповіді» — `cy.log()`. Для повного аналізу — `console.log()`. Для покрокової перевірки логіки — `debugger`.

18. Просунуті техніки

→ Перевірка швидкості запиту

🌟 ЖИВИЙ ПРИКЛАД

```
it('запит має бути швидшим за 150ms', () => {  
  cy.request(request).then(response => {  
    assert.isTrue(response.duration <= 150);  
  });  
});
```

→ Рандомізація даних

Іноді треба генерувати унікальні значення (напр., щоб не наразитись на «такий юзер вже існує»):

🌟 ЖИВИЙ ПРИКЛАД

```
function getRandomInt(max) {  
  return Math.floor(Math.random() * max);  
}  
  
it('10 запитів з рандомними ID', () => {  
  for(let i = 0; i < 10; i++) {  
    const randomId = getRandomInt(10000000);  
    cy.request({  
      url: 'https://api.com/users/' + randomId  
    }).then(resp => {  
      assert.isTrue(resp.status == 200);  
    });  
  }  
});
```

→ User-Agent і Cookies — як заголовки

🌟 ЖИВИЙ ПРИКЛАД

```
const request = {  
  url: 'https://httpbin.org/headers',  
  headers: {  
    'user-agent': 'My test bot 1.0',  
    'Cookie': 'sessionId=abc123'  
  }  
};
```

⚠ **ТИПОВА ПОМИЛКА**

Бібліотека `faker.js` — кращий вибір для генерації імен, email, адрес тощо. Не пиши свої «костилі», коли є готова бібліотека.

Частина 5. Artillery – навантажувальне тестування

19. Що таке Artillery

Artillery — це інструмент для перевірки, як сервер витримує навантаження. Можна симулювати сотні віртуальних користувачів і дивитись, чи не падає система.

→ Що перевіряє нав. тестування

Метрика	Що означає
Speed	Як швидко сервер відповідає
Stability	Чи стабільно працює тривалий час
Reliability	Скільки помилок повертає
Scalability	Як справляється зі зростанням навантаження
Responsiveness	Як змінюється швидкість при змінах нав.

→ Типи навантажувальних тестів

Тип	Простими словами
Load	Звичайне очікуване навантаження
Stress	Більше за очікуване — шукаємо «межу»
Scalability	Поетапне збільшення навантаження
Spike	Раптовий пік (як ефект Чорної п'ятниці)
Volume	Багато даних у системі
Endurance	Довгий тест — шукаємо memory leaks

→ Чому Artillery

- ▶ Працює на Node.js (як і Cypress)
- ▶ Тести у форматі **YAML** — без коду
- ▶ Підтримує HTTP та WebSocket
- ▶ Купа плагінів і можливість додавати JS-код

20. Встановлення Artillery

→ Передумова — Node.js

```
$ node -v  
npm -v
```

→ Глобальна установка Artillery

```
$ npm install -g artillery
```

→ Перевірка

```
$ artillery dino
```

Якщо побачив динозаврика з ASCII-графіки — все ок 🦖

→ Запуск тесту

```
$ artillery run test.yml
```

⚠ ТИПОВА ПОМИЛКА

YAML-формат дуже чутливий до відступів і пробілів. Один зайвий пробіл — і тест не запуститься. Використовуй валідатор: yamllint.com

21. Перший Artillery-тест

Файл тесту складається з **двох частин**: `config` (налаштування) та `scenarios` (що робить юзер).

🌟 ЖИВИЙ ПРИКЛАД

Файл `test.yml`:

```

config:
  target: "http://httpbin.org/"
  phases:
    - duration: 1
      arrivalRate: 1
      name: Artillery First Test

scenarios:
  - name: Getting a resource
    flow:
      - get:
          url: "{{ target }}get"

```

→ Розбираємо config

- ▶ `target` — базова адреса (хост)
- ▶ `phases` — фази тесту:
 - ▶ `duration: 1` — тривалість 1 секунда
 - ▶ `arrivalRate: 1` — додавати 1 юзера за секунду
 - ▶ `name` — ім'я фази

→ Розбираємо scenarios

- ▶ `name` — назва сценарію
- ▶ `flow` — послідовність дій юзера
- ▶ `{{ target }}` — підстановка змінної з config

→ Запуск і метрики

```
$ artillery run test.yml
```

Метрика	Що означає
<code>http.codes.200</code>	Скільки запитів повернули 200
<code>http.request_rate</code>	Середня кількість запитів за секунду
<code>http.requests</code>	Загальна кількість запитів
<code>http.response_time.p95</code>	95% юзерів отримали відповідь за цей час
<code>http.response_time.p99</code>	99% юзерів — за цей час
<code>vusers.failed</code>	Скільки юзерів не змогли пройти сценарій

 LIFEHACK

Метрика **p95** важливіша за **середню**. Середнє «маскує» довгі хвости — а p95 показує реальний досвід більшості користувачів.

22. POST-запити та capture

→ POST-запит з тілом

★ ЖИВИЙ ПРИКЛАД

```
scenarios:
  - name: Login flow
    flow:
      - post:
          url: "{{ target }}post"
          json:
            username: "Test User"
            password: "Temp Password"
```

💡 LIFEHACK

Маєш великий JSON, який треба перевести в YAML? Сервіс json2yaml.com зробить це за тебе.

→ Capture — зберігаємо дані з відповіді

Часто треба взяти токен з відповіді логіну і використати в наступних запитах. Для цього є `capture`.

★ ЖИВИЙ ПРИКЛАД

```
- post:
  url: "{{ target }}auth/login"
  json:
    username: "test@x.com"
    password: "123"
  capture:
    - json: "$.accessToken"
      as: "token"
    - json: "$.user.id"
      as: "userId"

# Використовуємо в наступному запиті
- get:
  url: "{{ target }}users/{{ user_id }}"
  headers:
    Authorization: "Bearer {{ token }}"
```

→ Як підглянути збережені значення

Команда `log` виводить значення в термінал:

🌟 ЖИВИЙ ПРИКЛАД

```
- log: "Token: {{ token }}"
- log: "User ID: {{ userId }}"
```

⚠️ ТИПОВА ПОМИЛКА

Знак `$` — це JSONPath. `$.json` — корінь JSON, `$.headers.Host` — вкладене поле. Без `$` capture не спрацює.

23. Плагін ensure — авто-валідація

`ensure` — вбудований плагін, який сам перевіряє метрики після тесту і пише  або .

→ Підключення

🌟 ЖИВИЙ ПРИКЛАД

```
config:
  plugins:
    ensure: {}
  ensure:
    thresholds:
      - http.response_time.p95: 1000
    conditions:
      - expression: vusers.failed == 0
      - expression: "http.request_rate >= 2 and http.response_time.max < 600"
```

→ Два типи перевірок

Тип	Що це
<code>thresholds</code>	Верхні порогові значення (типу «не повинно перевищувати»)
<code>conditions</code>	Складні умови (як assert) з операторами <code>and</code> , <code>>=</code> , <code>==</code>

💡 LIFEHACK

Складні вирази у `conditions` завжди бери в лапки. Прості (одна умова) — можна без лапок.

24. Типи тестів через секцію phases

Через різні комбінації `duration` + `arrivalRate` можна створити будь-який тип нав. тесту.

→ Load Test – звичайне навантаження

★ ЖИВИЙ ПРИКЛАД

```
phases:
- duration: 1800      # 30 хвилин
  arrivalRate: 2      # 2 юзери/сек = 120/хв
  name: Load Test
```

→ Stress Test – за межами норми

★ ЖИВИЙ ПРИКЛАД

```
phases:
- duration: 600      # 10 хвилин
  arrivalRate: 6      # у 3 рази більше за норму
  name: Stress Test
```

→ Scalability Test – поетапне зростання

★ ЖИВИЙ ПРИКЛАД

```
phases:
- { duration: 300, arrivalRate: 1, name: Phase 1 }
- { duration: 300, arrivalRate: 2, name: Phase 2 }
- { duration: 300, arrivalRate: 3, name: Phase 3 }
- { duration: 300, arrivalRate: 4, name: Phase 4 }
```

→ Spike Test – раптовий пік

★ ЖИВИЙ ПРИКЛАД

```

phases:
  - duration: 300
    arrivalRate: 2
    name: Warm-Up
  - duration: 10
    arrivalRate: 20      # РІЗКИЙ СПЛЕСК
    name: Spike
  - duration: 300
    arrivalRate: 2
    name: Cool-down

```

→ Endurance / Soak – на витривалість

★ ЖИВИЙ ПРИКЛАД

```

phases:
  - duration: 18000      # 5 годин
    arrivalRate: 1
    name: Soak Test

```

→ Додаткові параметри фаз

Параметр	Що робить
<code>arrivalCount</code>	Загальна кількість юзерів за <code>duration</code>
<code>rampTo</code>	Лінійне зростання навантаження
<code>maxVusers</code>	Максимум одночасних юзерів
<code>pause</code>	Пауза між фазами (без юзерів)

★ ЖИВИЙ ПРИКЛАД

```

# Лінійне зростання від 1 до 30 юзерів/сек за 60 секунд
phases:
  - duration: 60
    arrivalRate: 1
    rampTo: 30

```

⚠ **ТИПОВА ПОМИЛКА**

Не намагайся згенерувати 10000 юзерів з домашнього ноутбука на Wi-Fi — інтернет-канал не справиться. Для серйозного навантаження використовуй хмарні рішення (Artillery Cloud, BlazeMeter).

25. Просунуті фічі Artillery

→ Завантаження даних з CSV

Якщо тестуєш пошук — кожен юзер має використовувати **унікальний** кейворд (інакше система його кешує). На допомогу — `payload`.

🌟 ЖИВИЙ ПРИКЛАД

Файл `keywords.csv` :

```
Orange,McLaren
Apple,Lucid
Cherry,Ferrari
```

У `test.yml` :

```
config:
  payload:
    - path: "keywords.csv"
      fields:
        - "keyword1"
        - "keyword2"
      order: sequence

  scenarios:
    - flow:
      - get:
          url: "{{ target }}search?q={{ keyword1 }}"
```

→ Hooks – before / after сценарію

Хочеш авторизуватись один раз і використати токен у всіх сценаріях? Хуки тобі допоможуть.

🌟 ЖИВИЙ ПРИКЛАД

```
before:
  flow:
    - post:
        url: "{{target}}/auth/login"
        json:
          username: "test@x.com"
          password: "123"
        capture:
          - json: "$.accessToken"
            as: "accessToken"

after:
  flow:
    - get:
        url: "{{target}}/auth/logout"
        headers:
          Authorization: "Bearer {{ accessToken }}"
```

💡 LIFEHACK

Запити в `before` / `after` **не впливають** на метрики тесту. Авторизація і вихід з системи не «зіпсують» статистику.

→ DEBUG mode

Якщо щось не працює — запусти в дебаг-режимі і побачиш кожен запит/відповідь:

```
$ DEBUG=http artillery run test.yml
```

```
$ DEBUG=http,http:response artillery run test.yml
```

```
$ DEBUG=http* artillery run test.yml
```

На Windows: спочатку `set DEBUG=http`, потім запуск.

→ Processors — додаємо JS-код

Якщо вбудованих можливостей не вистачає — підключи свій JavaScript-файл.

🌟 ЖИВИЙ ПРИКЛАД

Файл `processor.js` :

```
function addCustomHeader(requestParams, context, ee, next) {
  requestParams.headers['X-Trace-Id'] = Date.now();
  return next();
}

module.exports = { addCustomHeader };
```

у `test.yml`:

```
config:
  processor: "processor.js"

scenarios:
  - flow:
      - get:
          url: "{{ target }}data"
          beforeRequest: "addCustomHeader"
```

→ Environments — різні оточення в одному файлі

Зручно: один файл — і local, і load, і spike-конфігурація.

🌟 ЖИВИЙ ПРИКЛАД

```
config:
  target: "http://httpbin.org/"
environments:
  local:
    phases:
      - { duration: 1, arrivalRate: 1, name: Debug }
  load:
    phases:
      - { duration: 1800, arrivalRate: 2, name: Load }
  spike:
    phases:
      - { duration: 10, arrivalRate: 20, name: Spike }
```

Запуск конкретного оточення:

```
$ artillery run -e local test.yml  
artillery run -e load test.yml  
artillery run -e spike test.yml
```

 LIFEHACK

Створи окреме оточення `local` з 1 юзером для дебагу. Не треба чекати 30 хвилин при кожній правці тесту.

Міні-шпаргалка — все в одному місці

Cypress — встановлення і запуск

Команда	Що робить
<code>npm init -y</code>	Створити проект
<code>npm install cypress --save-dev</code>	Встановити Cypress
<code>npx cypress open</code>	Відкрити GUI
<code>npx cypress run</code>	Запустити всі тести
<code>npx cypress run -b chrome --headed</code>	У Chrome з вікном
<code>npx cypress run --spec "path/to/test.cy.js"</code>	Один файл

Cypress — структура тесту

Конструкція	Призначення
<code>describe()</code>	Група тестів
<code>it()</code>	Окремий тест
<code>before()</code> / <code>after()</code>	Один раз до/після всіх
<code>beforeEach()</code> / <code>afterEach()</code>	До/після кожного
<code>.only()</code>	Тільки цей тест
<code>.skip()</code>	Пропустити

Cypress — команди

Команда	Опис
<code>cy.visit(url)</code>	Відкрити сторінку
<code>cy.get(selector)</code>	Знайти елемент
<code>cy.contains(text)</code>	Знайти за текстом

<code>.click()</code>	Клік
<code>.dblclick()</code>	Подвійний клік
<code>.rightclick()</code>	Правий клік
<code>.type(text)</code>	Ввести текст
<code>.clear()</code>	Очистити поле
<code>.submit()</code>	Відправити форму
<code>.scrollIntoView()</code>	Проскролити до елемента
<code>.should(assertion)</code>	Перевірка
<code>.and(assertion)</code>	Додаткова перевірка

Cypress – селектори

Тип	Приклад
Клас	<code>cy.get(".my-class")</code>
ID	<code>cy.get("#my-id")</code>
Тег	<code>cy.get("input")</code>
Атрибут	<code>cy.get("[type='email']")</code>
Best practice	<code>cy.get("[data-cy='login-btn']")</code>

Cypress – assertions

Assertion	Перевіряє
<code>be.visible</code>	Видимий
<code>exist</code> / <code>not.exist</code>	Існує
<code>have.text</code>	Текст
<code>have.value</code>	Значення інпуту
<code>have.class</code>	Клас
<code>have.css</code>	CSS-властивість
<code>be.checked</code>	Чекбокс відмічений

`be.disabled`

Заблокований

Cypress – API тестування

Команда / поле	Опис
<code>cy.request(url)</code>	Простий GET
<code>cy.request({...})</code>	Запит з налаштуваннями
<code>method</code>	HTTP-метод
<code>qs</code>	Query-параметри
<code>headers</code>	Заголовки
<code>body</code>	Тіло запиту
<code>failOnStatusCode: false</code>	Не падати на не-200
<code>response.status</code>	Код відповіді
<code>response.body</code>	Тіло відповіді
<code>response.headers</code>	Заголовки
<code>response.duration</code>	Тривалість (мс)

Artillery – головне

Команда	Опис
<code>npm install -g artillery</code>	Встановити
<code>artillery dino</code>	Перевірка
<code>artillery run test.yml</code>	Запустити тест
<code>artillery run -e load test.yml</code>	Запустити environment
<code>DEBUG=http artillery run test.yml</code>	Debug mode

→ Структура YAML-файлу

```

config:
  target: "http://api.com"
  phases: [...]
  plugins: { ensure: {} }
  payload: [...]
  processor: "file.js"
  environments: { ... }

before: # один раз перед
after:  # один раз після

scenarios:
- name: My scenario
  flow:
    - get: { url: "...", }
    - post: { url: "...", json: {...} }

```

→ Метрики Artillery

Метрика	Значення
<code>http.codes.200</code>	Кількість 200-х
<code>http.request_rate</code>	Запитів/сек
<code>http.response_time.p95</code>	95-й перцентиль
<code>http.response_time.p99</code>	99-й перцентиль
<code>vusers.failed</code>	Неуспішні юзери

Чекліст QA Automation Engineer

Перед написанням тестів

- ☐ Встановлено Node.js (`node -v`)
- ☐ Встановлено Cypress (`npm install cypress --save-dev`)
- ☐ Браузери Chrome / Firefox оновлені до останньої версії
- ☐ Налаштовано VSCode + термінал
- ☐ Створено структуру папок (e2e, pages, fixtures)

Якісний тест має

- ☐ Зрозумілу назву (що саме перевіряє)
- ☐ Один тест — одна перевірка / один сценарій
- ☐ Селектори типу `data-cy` (а не CSS-класи)
- ☐ Без залежності від попередніх тестів
- ☐ Без захардкоджених затримок (`cy.wait(5000)`) — тільки коли реально треба
- ☐ Повторювані частини винесені в custom commands або POP
- ☐ Має assertions (без них це не тест, а скрипт)

Перед комітом коду

- ☐ Прибрав `.only()` з усіх тестів
- ☐ Не залишилось закоментованого коду
- ☐ Тести проходять локально
- ☐ Селектори стабільні (не `.css-1f1fv1i`)
- ☐ `node_modules` у `.gitignore`
- ☐ Чутливі дані (паролі, API-ключі) в `.env` , а не в коді

API-тестування

- ☐ Покрив Happy Path (200 OK, валідні дані)
- ☐ Перевірив негативні кейси (400, 404, 401, 405, 500)
- ☐ Тестував з невалідними/відсутніми параметрами
- ☐ Тестував без авторизаційного токена
- ☐ Перевіряєш не лише `status`, а й `body`
- ☐ Час відповіді `response.duration` у межах SLA

Навантажувальне тестування

- ☐ Зрозумів очікуване навантаження від бізнесу
- ☐ Запустив Load Test з реальними рівнями
- ☐ Запустив Stress Test для пошуку межі
- ☐ Налаштував `ensure` з порогами SLA
- ☐ Створив окремий `local` environment для дебагу
- ☐ Прибрав `log` команди з продакшен-конфігу
- ☐ Документуєш результати після кожного запуску

Частина 7. CI/CD для QA Automation

28. Що таке CI/CD

CI/CD = **Continuous Integration / Continuous Deployment**. Простими словами — це набір підходів, щоб код від розробника швидко і безпечно доходив до користувача.

→ Розшифровка

Абревіатура	Що означає
CI	Continuous Integration — постійна інтеграція коду
CD	Continuous Delivery / Deployment — постійна доставка

→ Класичний CI/CD pipeline



i INFO

CI/CD стосується доставки **будь-якого** коду: backend, frontend і навіть твоїх автотестів. Принципи всюди однакові.

29. CI/CD для AQA — навіщо це нам

Написати тести — це лише пів справи. Далі їх треба запускати **автоматично**, а не лише в себе на лептопі.

→ Коли запускати автотести

Тригер	Що відбувається
Новий commit у main	Прогнати регресію після оновлення фронту/бека
Pull Request	Перевірити, що нова фіча нічого не зламала
Розклад (schedule)	Нічні прогони на повному наборі тестів
Ручний запуск	Перевірити перед релізом

💡 LIFEHACK

Нічні прогони — улюблене рішення команд. Прогнав 500 E2E-тестів за 3 години, прийшов уранці — побачив звіт. Лептоп вільний для роботи протягом дня.

30. Continuous Integration — етапи

CI відповідає за «перевірити, що код хороший». Якщо хоч один етап не пройшов — увесь pipeline валиться.

→ Етап 1. Підготовка (Setup)

- ▶ Запускається build server (часто це Docker-контейнер)
- ▶ Встановлюється Node.js / Java / Python — що треба для проєкту
- ▶ Завантажується код з GitHub / GitLab

→ Етап 2. Stylechecking (перевірка стилю коду)

- ▶ Лінери (ESLint, Prettier) перевіряють код на стандарти
- ▶ Імена змінних, форматування, відсутність `console.log()`

Для AQA-проєктів часто пропускають, але для бекенду/фронту — обов'язково.

→ Етап 3. Unit-тести

- ▶ Запускаються тести найменших одиниць коду
- ▶ Якщо хоч один впав — pipeline зупиняється

Для AQA-проєктів пропускається — тести і є самі по собі тестами 😊

→ Етап 4. Build (збірка)

- ▶ Код пакується в **артефакт** (artifact)
- ▶ Часто це **Docker-образ** — самодостатній «контейнер» з усіма залежностями
- ▶ Цей образ можна запустити на будь-якій машині

i INFO

Артефакт (artifact) — це готовий файл, який вже можна запустити. Для AQA — це Docker-образ, який містить усі тести і виконує їх при запуску.

31. Continuous Delivery — доставка коду

Коли CI відпрацював і код «здоровий» — треба доставити його туди, де ним користуватимуться. Це і є CD.

→ Куди доставляється код

Оточення	Опис
dev	Сервер для розробників
staging	Тестове оточення, копія прода
production	Реальні користувачі

 НА ЗАМІТКУ

Для AQA-проектів CD зазвичай не потрібен. Тести запустились — і все, нікуди їх «доставляти» не треба.

32. Інструменти CI/CD

Існує два загальні підходи: окремі системи або вбудовані в Git-сервіс.

→ Спеціалізовані системи

Інструмент	Особливості
Jenkins	Найвідоміший, дуже гнучкий, плагіни на все
TeamCity	Від JetBrains, зручний UI
CircleCI	Хмарний, простий старт
Bamboo	Від Atlassian, інтеграція з Jira

→ Вбудовані в Git-системи

Інструмент	Платформа
GitHub Actions	GitHub
GitLab CI	GitLab
Bitbucket Pipelines	Bitbucket

→ Що обрати

- ▶ Старт проекту → GitHub Actions / GitLab CI (просто, безкоштовно для невеликих репозиторіїв)
- ▶ Великий проєкт → Jenkins / TeamCity (більше гнучкості, кастомізації)

→ Простий приклад GitHub Actions

★ ЖИВИЙ ПРИКЛАД

Файл `.github/workflows/tests.yml` :

```
name: Run Cypress Tests

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  cypress-run:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: 20
      - run: npm ci
      - run: npx cypress run
```

💡 LIFEHACK

У GitHub Actions багато готових action'ів для Cypress: `cypress-io/github-action`. Не треба писати все з нуля.

→ Плюси та мінуси CI/CD — коротко

✓ Переваги	✗ Недоліки
Ранні баги — поламаний код не доходить до прода	Висока кваліфікація — налаштувати непросто
Швидка доставка фіч до користувачів	Витрати на інфраструктуру (build-сервери)
Менше рутинної роботи	Потрібен DevOps або відповідальна людина
Стабільність — авто-деплой не помиляється	Документація для команди (як це працює)
Конкурентна перевага	Час на впровадження

⚠ ТИПОВА ПОМИЛКА

Не намагайся з нуля побудувати ідеальний pipeline на 20 етапів. Це класичне «over-engineering» — половина не запрацює, а друга буде ламатись щотижня. Починай з малого!

Частина 8. BDD та Cucumber.io

33. Що таке BDD

BDD = **Behavior Driven Development** — підхід до розробки, де вимоги пишуться так, щоб однаково розуміли і бізнес, і розробка, і тестування.

→ Проблема, яку розв'язує BDD

- ▶ **Бізнес** розуміє, чого хоче, але не розуміє технічних викликів
- ▶ **Розробники** розуміють техніку, але можуть пропустити суть бізнес-задачі
- ▶ **Тестувальники** часто отримують вимоги останніми

BDD — це **міст** між цими ролями. Усі читають однакові вимоги і однаково їх розуміють.

→ Класичний vs BDD підхід

Класика — фокус «ЯК»	BDD — фокус «ЩО»
«Юзер вводить дані в текстове поле»	«Користувач вносить свої дані»
«Натискає кнопку Submit»	«Підтверджує введення»
Прив'язка до конкретного UI	Незалежно від реалізації

💡 LIFEHACK

BDD не каже розробці **як** робити — лише **що** має статися. Це звільняє розробників і не прив'язує до конкретного UI.

→ BDD + Agile + User Stories

BDD не замінює Agile — він **підсилює** його. Розглянемо приклад User Story і її BDD-варіант.

User Story (історія користувача):

🌟 ЖИВИЙ ПРИКЛАД

```
As an mobile bank customer
I want to see balance on my accounts
So that I can make better informed decisions
```

Та сама історія через Gherkin (BDD):

🌟 ЖИВИЙ ПРИКЛАД

Feature: Account balance visibility

Scenario: Do not show balance if not logged in

Given I am not logged on to the mobile banking app

When I open the mobile banking app

Then I can see a login page

And I do not see account balance

Scenario: Show balance after logging in

Given I have just logged on to the mobile banking app

When I load the accounts page

Then I can see account balance for each of my accounts

→ Ключові переваги BDD

- ▶ Усі вимоги відстежуються до бізнес-задач
- ▶ Розробка чітко знає, чого хоче бізнес
- ▶ Ефективна пріоритизація — критичні фічі першочергово
- ▶ Спільна термінологія між відділами (**Ubiquitous Language**)

i INFO

Ubiquitous Language — загальноприйнята мова команди. Суміш бізнес- і технічної мови, яку всі розуміють однаково. Унікальна для кожного проєкту.

34. Cucumber.io та Step Definitions

Cucumber — найпопулярніший BDD-фреймворк. Він читає сценарії звичайною мовою (Gherkin) і виконує їх як автотести.

→ Що робить Cucumber

- ▶ Структурує Gherkin-тести
- ▶ Зв'язує текстові кроки з кодом (через step definitions)
- ▶ Запускає тести і дає звіт

→ Step Definitions — місток між текстом і кодом

Кожен Gherkin-крок має мати свій **step definition** — це JS-функція, яка реалізує крок.

🌟 ЖИВИЙ ПРИКЛАД

Gherkin (.feature):

```
Scenario: Some cukes
```

```
  Given I have 48 cukes in my belly
```

Step Definition (.js):

```
const { Given } = require('@cucumber/cucumber');

Given('I have {int} cukes in my belly', function (cukes) {
  console.log(`Cukes: ${cukes}`);
});
```

Текст у лапках співпадає з Gherkin-кроком — так Cucumber розуміє, який код виконати для якого кроку.

`{int}` — параметр (число), який передається у функцію.

💡 LIFEHACK

Gherkin перекладений на 70+ мов, серед яких українська. Тобто можна писати `Дано`, `Коли`, `Тоді` замість Given/When/Then.

35. Gherkin – синтаксис

Gherkin — це не повноцінна мова, а підхід до формування сценаріїв. Базова структура: **Feature** → **Scenario** → **Steps**.

→ Ключові слова

Ключове слово	Призначення
Feature	Найвищий рівень — модуль або велика фіча
Scenario	Один тестовий сценарій
Given	Прекондишен (стартова умова)
When	Дія користувача
Then	Очікуваний результат
And / But	Додаткові кроки (для читабельності)
Background	Кроки, які виконуються перед кожним сценарієм
Scenario Outline	Сценарій з параметрами + Examples

→ Найпростіший сценарій

🌟 ЖИВИЙ ПРИКЛАД

```
Feature: Перша Фіча
  Scenario: Перший сценарій
    Given юзер вивчає автоматизацію
    When юзер стає розумніший кожного дня
    Then юзер отримує джоб оффер
```

→ 3 And / But — для читабельності

🌟 ЖИВИЙ ПРИКЛАД

Без And/But (повторюється Given/Then):

```
Given one thing
Given another thing
Given yet another thing
When I open my eyes
Then I should see something
Then I shouldn't see something else
```

3 And/But (краще):

```
Given one thing
And another thing
And yet another thing
When I open my eyes
Then I should see something
But I shouldn't see something else
```

→ Background – спільні кроки

Якщо в кожному сценарії однаковий **Given** — винось у **Background**. Він виконається перед **КОЖНИМ** сценарієм фічі.

🌟 ЖИВИЙ ПРИКЛАД

Feature: Login tests

Background:

Given temporary user is created

Scenario: User opens login page

Given user goes to "Login" page

Then login form is visible

Scenario: User logs in

When user enters credentials

Then user sees dashboard

⚠️ ТИПОВА ПОМИЛКА

У фічі може бути **тільки один** Background. Якщо різним сценаріям потрібні різні передумови — розбий на кілька фіч.

→ Scenario Outline – параметризований сценарій

Замість писати 5 однакових сценаріїв з різними значеннями — використовуй **Scenario Outline** + **Examples**.

★ ЖИВИЙ ПРИКЛАД

```
Scenario Outline: eating
  Given there are <start> cucumbers
  When I eat <eat> cucumbers
  Then I should have <left> cucumbers
```

Examples:

start	eat	left
12	5	7
20	5	15
100	50	50

Cucumber запустить сценарій 3 рази — по одному для кожного рядка таблиці.

→ Tags (теги) – фільтрація запуску

Теги починаються з **@** і ставляться перед **Feature** або **Scenario**. Теги фічі автоматично застосовуються до всіх її сценаріїв.

★ ЖИВИЙ ПРИКЛАД

```
@login @regression
Feature: Login

@smoke
Scenario: Positive Login
  Given user logs in
```

Запуск тестів за тегом

```
$ npx cucumber-js --tags '@smoke'
npx cucumber-js --tags 'not @skip'
npx cucumber-js --tags '@login and @smoke'
```

→ Multiline Text – багаторядковий текст

Якщо у крок треба передати великий блок тексту — використовуй потрібні лапки.

★ ЖИВИЙ ПРИКЛАД

```
Given user enters name and password
```

```
"""
```

```
GoIT Cucumber JS Introduction
```

```
Topic - Multiline Text
```

```
"""
```

→ Data Tables – таблиці даних у кроках

Якщо в одному кроці треба передати кілька об'єктів — додай таблицю прямо в крок.

★ ЖИВИЙ ПРИКЛАД

```
Given courses are created
```

```
| coursename | technology |
```

```
| Course1    | QA         |
```

```
| Course2    | AQA        |
```

💡 LIFEHACK

У step definition таблицю можна перетворити на масив об'єктів через метод `dataTable.hashes()`.

Дуже зручно для роботи з даними!

36. Встановлення Cucumber

→ Передумова

```
$ node -v  
npm -v
```

→ Покрокова установка

1. Створи папку проєкту, наприклад `cucumber-tests`
2. Зайди в неї і ініціалізуй проєкт:

```
$ npm init --yes
```

3. Встанови Cucumber як dev-залежність:

```
$ npm install --save-dev @cucumber/cucumber
```

4. У файлі `package.json` у секції `scripts` додай:

```
"test": "cucumber-js"
```

5. Створи структуру папок:

```
features/  
└─ step_definitions/
```

6. Створи `cucumber.js` у корені проєкту:

```
module.exports = {  
  default: '--format-options \'{ "snippetInterface": "synchronous" }\''  
}
```

7. Створи `features/step_definitions/stepdefs.js`:

```
const assert = require('assert');  
const { Given, When, Then } = require('@cucumber/cucumber');
```

→ Перевірка установки

```
$ npm test
```

Якщо все ок — побачиш повідомлення:

```
0 Scenarios
0 steps
0m00.000s
```

Це означає, що Cucumber встановлений правильно і не знайшов тестів (бо ми ще їх не написали).

37. Перший Cucumber тест

Напишемо тест за принципом: «Чи сьогодні п'ятниця?». Класичний приклад з документації.

→ Крок 1. Створюємо .feature файл

У папці `features/` створи `is_it_friday.feature`:

★ ЖИВИЙ ПРИКЛАД

```
Feature: Is it Friday yet?
  Everybody wants to know when it's Friday

Scenario: Sunday isn't Friday
  Given today is Sunday
  When I ask whether it's Friday yet
  Then I should be told "Nope"
```

→ Крок 2. Запускаємо — Cucumber пропонує snippets

```
$ npm test
```

Cucumber побачить «не визначені кроки» і **сам** запропонує тобі шаблони коду для них. Зручно — копіюй і вставляй у `stepdefs.js`.

→ Крок 3. Реалізуємо step definitions

★ ЖИВИЙ ПРИКЛАД

Файл `stepdefs.js`:


```
const assert = require('assert');
const { Given, When, Then } = require('@cucumber/cucumber');

function isItFriday(today) {
  if (today === "Friday") return "TGIF";
  return "Nope";
}

Given('today is Sunday', function () {
  this.today = 'Sunday';
});

When("I ask whether it's Friday yet", function () {
  this.actualAnswer = isItFriday(this.today);
});

Then('I should be told {string}', function (expectedAnswer) {
  assert.strictEqual(this.actualAnswer, expectedAnswer);
});
```

i INFO

Слово `this` у Cucumber — це **World**, спільний контекст одного сценарію. Зберігаєш у `this.x` у Given — використовуєш у Then.

→ Що відбувається при запуску

1. `Given today is Sunday` → встановлює `this.today = 'Sunday'`
2. `When I ask...` → викликає `isItFriday('Sunday')` → "Nope"
3. `Then I should be told "Nope"` → перевіряє, що результат = "Nope" ✓

38. Структурні підходи на практиці

→ Scenario Outline – багатократні запуски

Що, якщо хочемо перевірити різні дні? Не дублюємо сценарії — використовуємо `Scenario Outline`.

★ ЖИВИЙ ПРИКЛАД

Файл `is_it_friday.feature`:

```
Feature: Is it Friday yet?
```

```
Scenario Outline: Today is or is not Friday
```

```
  Given today is "<day>"
```

```
  When I ask whether it's Friday yet
```

```
  Then I should be told "<answer>"
```

```
Examples:
```

day	answer
Friday	TGIF
Sunday	Nope
anything else!	Nope

Step Definition оновлюється — приймає параметр `{string}`:

```
Given('today is {string}', function (givenDay) {  
  this.today = givenDay;  
});
```

Cucumber запустить сценарій **3 рази** — і всі 9 кроків повинні пройти.

→ Background – для всіх сценаріїв одразу

★ ЖИВИЙ ПРИКЛАД

```
Feature: Is it Friday yet?
```

```
Background:
```

```
  Given today is a first summer month
```

```
Scenario: Sunday is not Friday
```

```
  Given today is "Sunday"
```

```
  ...
```

```
Scenario: Friday is Friday
```

```
  Given today is "Friday"
```

```
  ...
```

Background-крок виконається перед **КОЖНИМ** з двох сценаріїв.

→ Теги – селективний запуск

★ ЖИВИЙ ПРИКЛАД

```
@first-test
```

```
Scenario: Sunday is not Friday
```

```
  ...
```

```
@second-test
```

```
Scenario: Friday is Friday
```

```
  ...
```

Запуск:

```
$ npx cucumber-js --tags '@first-test'  
npx cucumber-js --tags 'not @first-test'
```

→ Data Tables – таблиці у кроках

Найпотужніша фіча Cucumber — передавати в крок цілі таблиці даних.

★ ЖИВИЙ ПРИКЛАД

Feature:

Feature: Shopping Cart

Scenario: Calculate total cost

Given the following products are added to the shopping cart:

Product	Price
Laptop	1000
Headphones	100
Mouse	20

When I calculate the total cost

Then the total cost should be 1120

Step Definition:

```
let shoppingCart = [];  
  
Given('the following products are added to the shopping cart:',  
  function (dataTable) {  
    shoppingCart = dataTable.hashes();  
    // → [{ Product: 'Laptop', Price: '1000' }, ...]  
  });  
  
When('I calculate the total cost', function () {  
  let total = 0;  
  for (const p of shoppingCart) {  
    total += parseInt(p.Price);  
  }  
  this.totalCost = total;  
});  
  
Then('the total cost should be {int}', function (expected) {  
  assert.strictEqual(this.totalCost, expected);  
});
```

💡 LIFEHACK

Метод `dataTable.hashes()` перетворює таблицю на **масив об'єктів**, де перший рядок таблиці = ключі. Це надзвичайно зручно для роботи з даними.

🎯 Міні-шпаргалка — Частина 2

CI/CD — ключові поняття

Термін	Що означає
CI	Continuous Integration — постійна інтеграція
CD	Continuous Delivery / Deployment
Pipeline	Послідовність етапів CI/CD
Build	Збірка коду в готовий артефакт
Artifact	Готовий файл для запуску (часто Docker-образ)
Trigger	Подія, яка запускає pipeline (push, PR, schedule)

CI/CD — найпопулярніші інструменти

Інструмент	Тип
Jenkins	Окремий сервер
GitHub Actions	Вбудовано в GitHub
GitLab CI	Вбудовано в GitLab
CircleCI	Хмарний
TeamCity	Окремий сервер (JetBrains)

Gherkin — ключові слова

Ключове слово	Призначення
Feature	Модуль / велика фіча
Scenario	Один тест
Scenario Outline	Параметризований тест
Examples	Таблиця значень для Outline

Background	Спільні кроки перед кожним сценарієм
Given	Прекондишен (вихідний стан)
When	Дія
Then	Очікуваний результат
And / But	Додаткові кроки для читабельності
@tag	Тег для фільтрації

Cucumber — команди

Команда	Опис
<code>npm install --save-dev @cucumber/cucumber</code>	Встановити
<code>npm test</code>	Запустити тести (через скрипт)
<code>npx cucumber-js</code>	Запустити всі тести
<code>npx cucumber-js --tags '@smoke'</code>	Тільки з тегом
<code>npx cucumber-js --tags 'not @skip'</code>	Виключити тег
<code>npx cucumber-js features/login.feature</code>	Один файл

Cucumber — параметри в кроках

Параметр	Що приймає	Приклад Gherkin
<code>{string}</code>	Рядок у лапках	<code>"hello"</code>
<code>{int}</code>	Ціле число	<code>42</code>
<code>{float}</code>	Число з плаваючою комою	<code>3.14</code>
<code>{word}</code>	Одне слово (без лапок)	<code>hello</code>

Чекліст — Частина 2

CI/CD — впровадження

- ☐ Налаштовано базовий workflow на push до main
- ☐ Тести запускаються автоматично
- ☐ Налаштовано тригер для PR
- ☐ Звіти доступні команді (хоча б через email/Slack)
- ☐ Документація для команди — як працює pipeline
- ☐ Прибрано sensitive data в secrets/env variables
- ☐ Build-час не перевищує 30 хвилин (для зручності)

CI/CD — для AQA проекту

- ☐ Тести запускаються в чистому Docker-середовищі
- ☐ Налаштовано retry для нестабільних тестів
- ☐ Скріншоти/відео зберігаються як артефакти
- ☐ Налаштовано параметри запуску (browser, env)
- ☐ Падіння тесту → нотифікація відповідальному
- ☐ Окремі jobs для smoke / regression

BDD / Cucumber — якісний сценарій

- ☐ Назва Feature/Scenario зрозуміла бізнес-аналітику
- ☐ Кроки описують **поведінку**, а не реалізацію
- ☐ Без технічних деталей у Gherkin (ID, селекторів тощо)
- ☐ Один Then = один результат
- ☐ Використано And/But для уникнення повторів
- ☐ Background для спільних передумов

- ☐ Scenario Outline для тестів з різними даними
- ☐ Теги для класифікації (smoke, regression, skip)

BDD / Cucumber – структура проєкту

- ☐ Папка `features/` створена
- ☐ Step definitions у `features/step_definitions/`
- ☐ Конфіг `cucumber.js` у корені
- ☐ Скрипт `npm test` працює
- ☐ Cucumber та залежності в `package.json`
- ☐ Step definitions перевикористовуються між фічами
- ☐ `.gitignore` містить `node_modules`

\$ exit 0

Шпаргалка завершена.

Тепер у тебе є все, щоб почати писати круті автотести! 🚀

► Cypress ► API Testing ► Page Object
► Artillery ► CI/CD ► BDD / Cucumber

Made with ❤️ for QA Automation Engineers
Happy testing!